Jacob Yankee
COSC 341
Zhang

I. Reference
    A. No references were used.
II. Design
    A. For some reason, I decided to make all of my variables capital letters. I'm not sure why, maybe some things from prolog got mixed in with the lisp stuff in my brain.
III. Difficulty
    A. Could not figure out how to write insea in two functions or without using built in functions.
IV. Notes
    A. Would've been nice to start learning lisp at week nine or ten instead of week thirteen but that's just me.

Appendix One - Source Code

```
;Jacob Yankee
;COSC 341
;Zhang
;Homework 4

;Function 1
;Function name: disnpth
;Description: displays the n-th element of a list
;Notes: n/a
(defun dispth(L N)
  (if (eql n 1)
    (first L)
    (dispth (rest L) (- N 1))))

;Function 2
;Function name: delth
;Description: deletes the n-th element of a list
;Notes: n/a
(defun delth( L N)
  (if (null L) nil
    (if (eql N 1)
      (delth (rest L) (- N 1))
      (cons (first L) (delth (rest L) (- N 1))))))

;Function 3
;Function name: remv
```

```
;Description: removes elements from a list
;Notes: needed for function 4
(defun remv (K L)
  (if (null L) nil
    (if (eql K (first L))
      (remv K (rest L))
      (cons (first L) (remv K (rest L))))))

;Function 4
;Function name: remvdub
;Description: removes duplicate elements from a list
;Notes: calls function 3
(defun remvdub (L)
  (if (null L) nil
  (cons (first L) (remvdub (remv (first L) (rest L))))))

;Function 5
;Function name: min2
;Description: computes second smallest number of an integer list
;Notes: who approved of this syntax
(defun min2 (L)
  (if (null L) 0
      (if (null (rest L)) (first L)
          (if (null (rest (rest L)))
              (first (rest L))
              (if (< (first l) (first (rest L)))
                  (if (< (first (rest L)) (first (rest (rest L))))
                      (min2 (cons (first L) (cons (first (rest L)) (rest (rest (rest L))))))
                      (min2 (cons (first L) (rest (rest L)))))
                  (min2 (cons (first (rest L)) (cons (first L) (rest (rest L)))))))))

;Function 6
;Function name: helper
;Description: helper function for inde
;Notes: necessary for inde to have the input of a number and a list
(defun helper (N I L)
  (if (null L) nil
    (if (eql (first L) N)
      (cons I (helper N (+ I 1) (rest L)))
      (helper N (+ I 1) (rest L)))))

;Function name: inde
;Description: returns the index of occurence of a given value
;Notes: calls helper
```

```
(defun inde (N L)
  (helper N 1 L))

;Function 7
;Function name: helper
;Description: helper function for nele
;Notes: Necessary for nele to have the input of a number and a list
(defun helper (X Y L)
  (if (null L) nil
  (if (eql Y 0)
    (helper X X (rest L))
    (cons (first L) (helper X (- Y 1) L)))))

;Function name: nele
;Description: repeats each element in a list n times
;Notes: calls helper
(defun nele (L N)
  (helper N N L))

;Function 8
;Function name: smap
;Description: maps elements
;Notes: it's smap
(defun smap (F L)
  (if (null L) nil
    (cons (funcall F (first L)) (smap F (rest L)))))

;Function name: infront1
;Description: inserts an element as the head of each element of a list
;Notes: higher order, needs smap
(defun infront1 (N L)
  (smap (lambda (x) (cons N X)) L))

;Function 9
;Function name: insEvery
;Description: helper function to insert at every position
;Notes: has a labels function insAt which inserts at, may or may not be allowed
(defun insEvery (X L)
  (labels ((insAt (X N L)
           (if (> N (length L))
               nil
               (cons (insHelp X N L)
                     (insAt X (+ N 1) L)))))
    (insAt X 0 L)))
```

```
;Function name: insHelp
;Description: helper Function
;Notes: n/a
(defun insHelp (X N L)
  (if (eql N 0)
      (cons X L)
      (cons (first L) (insHelp X (- N 1) (rest L)))))

;Function name: insea
;Description: inserts an element to each position of a list
;Notes: this didn't go very well
(defun insea (X L)
  (insEvery X L))

;Function 10
;Function name: mergeL
;Description: helper for mergesort, merges lists
;Notes: called mergeL because merge is already a built in Function it seems
(defun mergeL (M N)
  (if (null M) N
      (if (null N) M
          (if (< (first M) (first N))
              (cons (first M) (mergeL (rest M) N))
              (cons (first N) (mergeL M (rest N)))))))

;Function name: split
;Description: helper for mergesort, splits lists
;Notes: works with let or let*
(defun split (L)
  (if (null L)
      (list nil nil)
      (if (null (rest L))
          (list (list (first L)) nil)
          (let ((splitL (split (rest (rest L)))))
            (list (cons (first L) (first splitL))
                  (cons (first(rest L)) (first(rest splitL))))))))

;Function name: mergesort
;Description: performs merge sort
;Notes: relies on above functions to use despired input syntax, let* works but let doesn't
(defun mergesort (L)
  (if (null L)
      nil
```

```
    (if (null (rest L))
        (list (first L))
        (let* ((splitL (split L))
               (X (first splitL))
               (Y (first (rest splitL))))
          (mergeL (mergesort X) (mergesort Y))))))

;Function 11
;Function name: app
;Description: append helper function
;Notes: n/a
(defun app (L X)
  (if (null L) X
  (cons (first L) (app (rest L) X))))

;Function name: nums
;Description: helper function to create a list
;Notes: n/a
(defun nums (N)
  (if (eql N 1) nil
    (app(nums (- N 1)) (list N))))

;Function name: fil
;Description: filter function
;Notes: I only named it fil instead of filter because that was its name in the notes
(defun fil(P L)
  (cond ((null L) NIL)
    ((funcall P (first L)) (cons (first L) (fil P (rest L))))
    (t (fil P (rest L)))))

;Function name: divisible
;Description: helper function for isPrime
;Notes: n/a
(defun divisible (N D)
  (if (eql D 1)
    t
    (if (eql (mod N D) 0) NIL
      (divisible N (- D 1)))))

;Function name: isPrime
;Description: helper function for primes
;Notes: n/a
(defun isPrime (N)
  (if (eql N 2)
```

```
    t
    (divisible N (- N 1))))

;Function name: primes
;Description: find all prime numbers from 2 to n
;Notes: higher order, also needs like a bunch of functions
(defun primes (N)
  (fil #'isPrime (nums N)))
```

Appendix Two - Sample Runs

Function 1:

Input:
(print (dispnth ‘(1 (2 3) 4 5) 2))

Output:
(2 3)

Function 2:

Input:
(print (delnth ‘(1 2 (3 4) 5) 3))

Output:
(1 2 5)

Function 3:

Input:
(print (remv 1 ‘(1 (2) 1 3)))

Output:
((2) 3)

Function 4:

Input:
(print (remvdub ‘(1 2 1 3 2 1))

Output:
(1 2 3)

Function 5:

Input:
(print (min2 ‘(1 3 2 5 4))

Output:
2

Function 6:

Input:
(print (inde 1 ‘(1 2 1 1 2 2 1))

Output:
(1 3 4 7)

Function 7:

Input:
(print (nele ‘(1 3 5) 3))

Output:
(1 1 1 3 3 3 5 5 5)

Function 8:

Input:
(print (infront1 1 ‘((1 2) nil (3)))

Output:
((1 1 2) (1) (1 3))

Function 9:

Input:
(print (insea 4 '(1 2 3)))

Output:
((4 1 2 3) (1 4 2 3) (1 2 4 3) (1 2 3 4))

Function 10:

Input:
(print (mergesort ‘(5 3 2 11 7))

Output:
(2 3 5 7 11)

Function 11:

Input:
(print (primes 20))

Output:
(2 3 5 7 11 13 17 19)